

Práctica 8: Análisis de Imágenes basado en ML – Detección de Objetos

Apellidos, nombre	SOLUCIÓN
-------------------	----------

El objetivo de esta práctica es presentar de forma práctica al alumno algunas de las técnicas de detección de objetos en imágenes vistas en teoría.

Desarrolle cada ejercicio en un fichero de comandos ‘P8_ejercicio.m’ separado y anote las observaciones que se le pidan en el guion. Para conocer el funcionamiento preciso de los comandos que se introducen en este guion, utilice la ayuda de MATLAB. Para evitar posibles interferencias con otras variables o ventanas recuerde incluir siempre las instrucciones

```
clear all;  
close all;  
clc;
```

al principio de cada fichero de comandos.

Al finalizar la práctica, comprima el guion con las observaciones y los ficheros ‘.m’ generados en un único fichero con el nombre ‘TSV_P8_ApellidosNombre.zip’, conéctese al sistema de entrega de prácticas de Moodle y entréguelo.

NOTA IMPORTANTE: En el desarrollo de esta práctica y posteriores se pide realizar operaciones repetidamente, por ejemplo, asignando individualmente píxeles concretos de una imagen. En estos casos se recomienda acudir al uso de estructuras de control (bucles, condiciones, etc.). Adicionalmente, debido a la creciente complejidad de los programas que se le va a pedir desarrollar, se recomienda encarecidamente el uso del depurador o *debugger* de MatLab, cuya funcionalidad está descrita en la ayuda del programa (*User's Guide - Desktop Tools and Development Environment - Editing and Debugging MATLAB Code*).

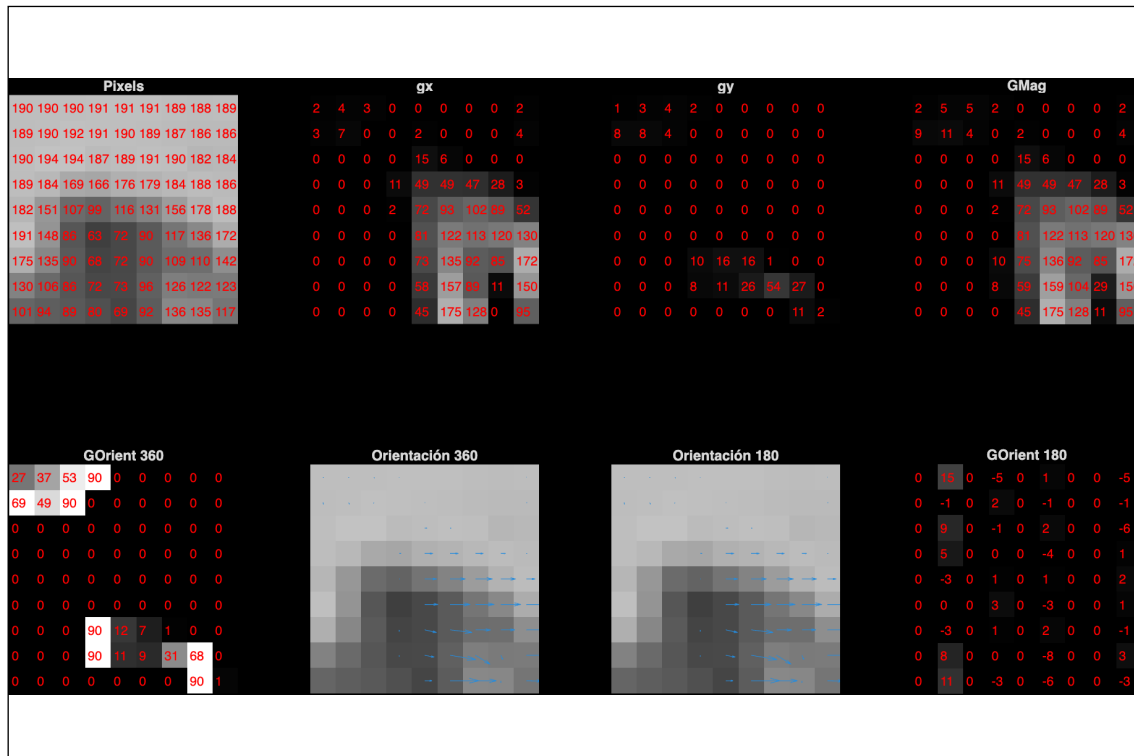
1. Detección de Objetos

1.1. Ejercicio 1. Segmentación mediante partición por nivel de gris

Complete el script P8_ejercicio1_template.m para visualizar los gradientes, su magnitud, su orientación 360° y su orientación 180°, de un bloque de 8x8 píxeles. La orientación 0° se corresponde con el eje positivo x y con sentido horario (0° a las 3, 90° a las 6, ...). Calcule el gradiente mediante los kernels

$dx = [-1 \ 0 \ 1; -1 \ 0 \ 1; -1 \ 0 \ 1]; \ dy = [-1 \ -1 \ -1; 0 \ 0 \ 0; 1 \ 1 \ 1];$

Guarde el script como P8_ejercicio1.m



1.2. Ejercicio 2. HOG de Matlab

Ejecute el script `P8_ejercicio2_template.m`, y comente las diferencias entre las distintas parametrizaciones de `extractHOGFeatures`

La default usa celdas 8x8, con celdas de 2x2 en cada bloque, 9 bins en el histograma y un rango $[0, \pi]$

La default RGB usará exactamente lo mismo, pero para una imagen con 3 canales en lugar de uno.

La SignedOrientation tiene en cuenta el rango $[0, 2\pi]$, lo que implica que ahora tiene en cuenta el efecto de la orientación del ángulo (hacia arriba / hacia abajo). Mientras que en default dos orientaciones 10° y 190° caían en el mismo bin, ahora no. Además, en default los bins cubrían un rango menor de valores (es fácil notar que si el número de bins no cambia, en SignedOrientation deberán cubrir rangos con el doble de valores).

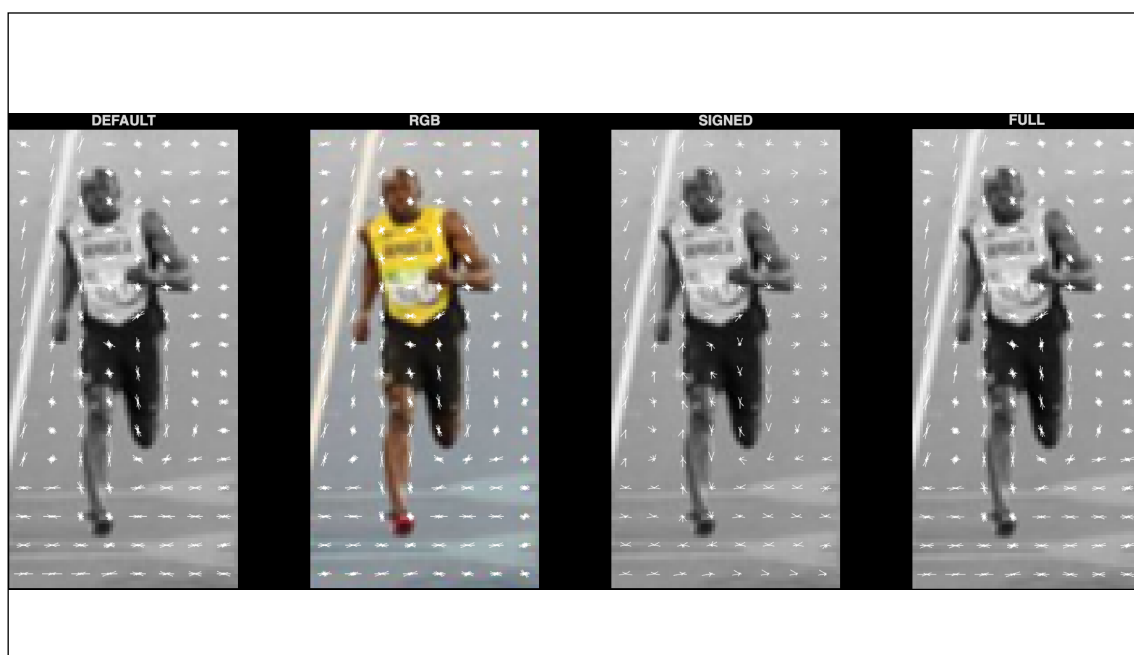
Los Dalal-Triggs HOG:

- cellsize -> tamaño de las celdas a normalizar
- blocksize -> número de celdas que se normalizan juntas
- BlockOverlap -> "patrón" que se sigue para comparar celdas dentro de bloques.

Al no usar SignedOrientation, lo único que cambia con respecto a default es BlockOverlap, que define `[1,1]`. Pero como esto es exactamente lo que se define por defecto en matlab, deberíamos obtener el mismo resultado,

Amplie la comparativa comparando la versión DEFAULT y la SIGNED, pero haciendo uso de 18 bins (no se puede comparar si no se pone en ambas el mismo número de bins, ya que generaría descriptores de diferente tamaño).

Guarde el script como `P8_ejercicio2.m`



--

1.3. Ejercicio 3. Detección Dalal-Triggs

Complete el script `P8_ejercicio3_template.m` para desarrollar un detector estilo Dalal-Triggs. (como se verá, funciona bastante “regular”). Entienda los diversos pasos que viene ya programados.

¿A qué puede deberse, entre otras causas, el funcionamiento “regular”?

Al haber tan pocas imágenes (800, 400 positivos, 400 negativos) de entrenamiento, SVM no generaliza bien. A mi parecer, el deslizamiento de la ventana deslizante es demasiado pequeño, por lo que genera muchos solapes. Además, el umbral es demasiado pequeño, de nuevo culpa del tamaño del dataset. Si tuviéramos un dataset más grande podríamos ampliar el umbral sin arriesgarnos a no clasificar peatones en ningún caso, además de hacerlo con más certeza como consecuencia precisamente de subir el umbral.

Guarde el script como `P8_ejercicio3.m`

